ENG2040 Assignment 1: Marketplace Orders to a Model-Ready Dataset

Noel Thomas (25011152)

October 2026

Contents

Summary 1

1 Loading the data 2

2 Data quality and cleaning 3

3 Exploratory analysis 4

3.1 Q1. How often are orders late, and by how much? . . . 4

3.2 Q2. Time from approval to the estimate . . . 4

3.3 Q3. Which states, and where to look first? . . . 4

3.4 Data quality, sensitivity and confidence . . . 5

4 Feature engineering 5

5 Reproducibility and resources 6

Summary

- All 20,000 orders are kept: 16,482 get a reliable label, the rest a reason why not. A value is fixed only when an independent record (a *witness*) shows the true value.
- 6.7% of labelled orders are late (95% interval 6.3–7.1%). Lateness comes in waves, short estimates are late twice as often, and RJ adds the most late deliveries.
- 16,482 × 72 numeric features, only from what is known at approval. One command rebuilds and checks every output.

Key decisions.

decision	instead of	because
SQL in SQLite via ADBC	pandas, DuckDB, Polars	least memory at every scale (§1)
Parquet with zstd	CSV, snappy, gzip	smallest at the same speed, keeps types (§1)
fix only with a witness	guessing or dropping	fixes match payments like clean orders (§2)
keep every order, give a reason	dropping unusable orders	every record counted (§2, §3)
Wilson intervals	textbook $p \pm 1.96\,\mathrm{se}$	valid for small states, e.g. 0 of 8 (§3)
rank by excess late orders	by late rate	small states cannot dominate (§3)
missing as NaN + flag	filling with a median	a fill now would use future test orders (§4)
time-split check model	random split	learn from the past, predict the future (§4)

Data: Olist extract, modified for the course [10]. Code: `pipeline.ipynb`, `sql/`, `bench.py`, `run.py`. The README maps each step to its inputs, outputs and row meaning, and each brief question to where it is answered.

1 Loading the data

table	rows	key	one row is
orders	20,000	order_id	an order (the grain)
customers	20,000	customer_id	the customer record of one order
order_items	22,662	order_id + order_item_id	one item (2 of a product = 2 rows)
order_payments	20,875	order_id + payment_sequential	one payment, in centavos
order_reviews	19,966	review_id + order_id	one review
products / sellers	11,334 / 2,071	product_id / seller_id	catalogue rows

One row per order.

- Items, payments and reviews are one-to-many. Joined raw, 3 items × 2 payments = 6 rows and double-counted totals.
- So `sql/orders.sql` aggregates each per order inside SQLite, then left joins onto orders. No fan-out, no order lost. Payments stay in integer centavos, so totals are exact.
- `EXPLAIN QUERY PLAN` [3]: each GROUP BY walks the `order_id` index.

Relationships and totals preserved (`key_links.csv`, `checks.csv`).

- 0 items, payments or reviews point to a missing order.
- Gaps the other way stay as NULLs: 769 orders have no customer row, 142 have no items, 910 items have no product row and 893 have no seller row.
- 0 duplicate orders, 0 customer records shared by two orders; the 998 repeated item rows are quantity. Counts and totals (price total 54,446,205.77, payment total 280,524,473 centavos) are equal in the database and the output.

Why this is the most efficient choice. The same table built 8 ways at 1×, 3× and 10× the data, each in a fresh process, median of 3 (`bench.py`).

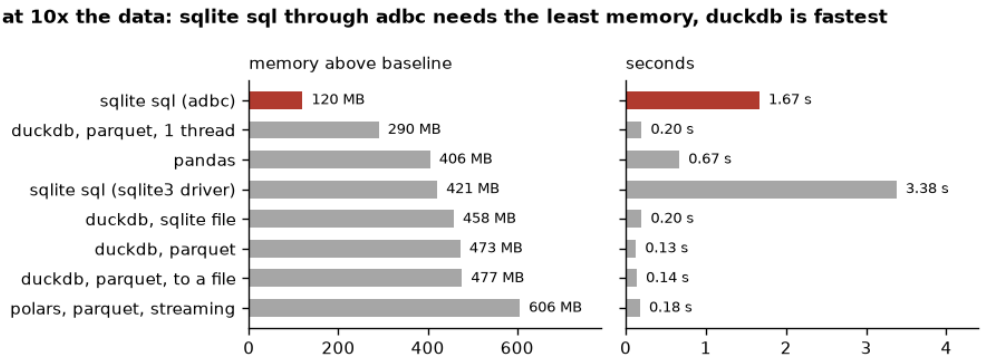


Figure 1: At 10× the data, SQLite SQL through ADBC needs the least memory. DuckDB is fastest but needs 2.4–4× the memory.

- **My first estimate was wrong.** With Python's `sqlite3` driver, SQL needed as much as pandas (421 vs 406 MB): it builds a Python object per value, 20 MB of objects for a result that is 7 MB as a dataframe. ADBC hands over Arrow columns [4], and SQL then needs under a third of pandas' memory (120 vs 406 MB).
- **DuckDB** [5] is 5–13× faster but holds 290–480 MB for its join hash tables. Writing to a file did not help; a memory cap made the query fail.
- **Polars** (lab 7), streaming [6]: the most memory (606 MB). Streaming helps when inputs do not fit in memory; here they do.
- **Decision:** no engine wins on both. Memory decides whether a step fits at all; 1.7 s at 10× is fine for a batch step.
- **Storage:** Parquet keeps types that CSV turns into text [7]. Orders: CSV 5.25 MB, Parquet uncompressed 4.66, snappy 3.29, gzip 2.15, zstd 2.08. zstd is smallest at snappy's speed, so all outputs use it (`storage.csv`).

2 Data quality and cleaning

Rule, strongest first. Fix only with a witness; else **set missing**; **flag** fields that feed neither labels nor features. No order is deleted. Every change is logged with key, column, rule, old and new value (`changes.parquet`).

problem	values	action
negative price / freight	408 / 458	absolute value
price > 5,000 / freight > 500	417 / 473	divide by 1,000
size > 200 cm or weight > 100 kg	783 items	missing
−1 photos, name or description length	690 items	missing
state ZZ or blank	838 orders, 1,031 items	missing, then recovered from zip
impossible date, size ≤ 0, placeholder category	see `profile.csv`	missing
status with spaces or capitals	783	strip, lowercase

Why these fixes are trusted.

- **Prices.** The payment is a witness. After the fix, price + freight matches the payment for 87% / 90% (negative) and 94% / 88% (large) of orders, the same as clean orders (90%). Dividing by 100 matches 0%. A second witness: the same product sells elsewhere at the fixed price for 178 of 277.
- **Sizes.** Freight says the values are wrong: "over 100 kg" items ship for R$15.2, like 0.4–1.4 kg items (R$16.1), not 20–30 kg ones (R$74.6). But freight cannot give the exact value, so missing, not divided.
- **States.** A zip prefix sits in one state, so missing states come back from the zip (895 customers, 1,032 sellers). 14 prefixes with two states are not used. Recovered rows are flagged.

Labels. `delivered_after_estimate` = 1 if delivered after the estimated calendar date, 0 if on or before it (local clock). Only usable completed deliveries are labelled.

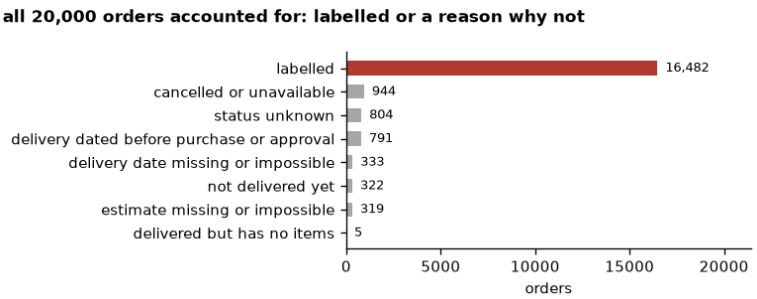


Figure 2: Every order is labelled or has one reason why not.

Unresolved, and what it affects.

- Price + freight now matches the payment for 16,302 of 18,122 complete orders (14,845 before). Of the 1,820 left, 919 have a negative payment and 832 paid exactly R$7.23 more.
- **R$7.23 extra** on 832 orders: same payment types, states and months as all orders (credit card 76% vs 72%, SP 40% vs 42%). Spread evenly, so it cannot bias a comparison.
- **Payments** (negative values, 929 instalments of −1) are left: they are only a witness. 0 instalments is valid.
- **Carrier pickup before approval** (270): flagged; the carrier date is not used.
- `not_reported` (804) and deliveries dated exactly 7 days before approval (782 of 791 too early): unlabelled, since a fix would be a guess. Section 3 puts them back as a test.
- **Extreme delays** (up to 188 days, 14 orders over 60): kept, their dates are consistent.

3 Exploratory analysis

Rates carry 95% Wilson intervals. The textbook $p \pm 1.96\,\mathrm{se}$ fails for small groups (0 late out of 8 gives 0–0%); Wilson gives 0–32% [1]. Brown, Cai and DasGupta recommend Wilson (or Jeffreys) for small samples [2].

3.1 Q1. How often are orders late, and by how much?

- 6.7% of labelled orders are late (6.3–7.1%), about 1 in 15.
- A late order is typically a week late (median 7 days, middle half 3–13); 1 in 8 is 3+ weeks late.
- The typical on-time order arrives 13 days early: estimates are padded.

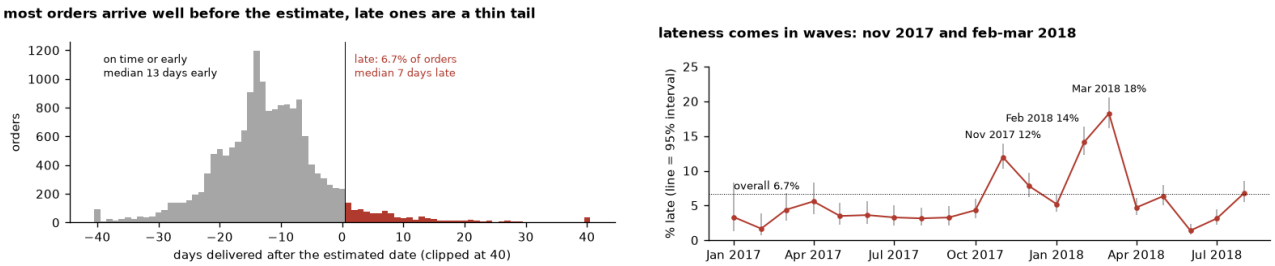


Figure 3: Lateness comes in waves, with 18% of March 2018 orders late against 1–8% in normal months.

3.2 Q2. Time from approval to the estimate

- Estimates under a week are late 16% of the time, twice any other group. From 1 to 5 weeks it is 5–7%; beyond 6 weeks, under 2%.
- **Not state or season:** no short estimate falls in the spike months, and within the same state and month, short estimates are late 16.3% vs 4.2%.
- An association, not a cause, but a short estimate is a useful risk signal at approval.

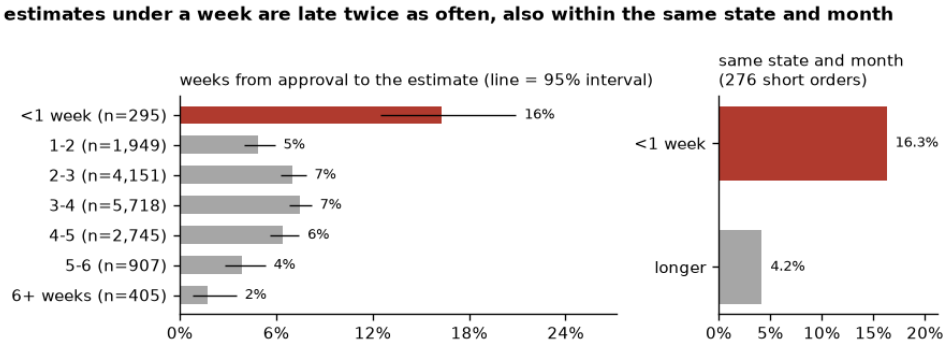


Figure 4: Short estimates are late twice as often, also within the same state and month.

3.3 Q3. Which states, and where to look first?

- Ranked by **excess late orders** = late orders − orders × overall rate: the late deliveries a state adds. A high rate on 9 orders is mostly noise; a slightly raised rate on 2,000 orders is many late deliveries.
- **RJ first** (+98), then BA (+26) and CE (+22). RJ's rate (11.5%) is not the highest, but it is on 2,036 orders.
- **Drill-down:** 65 of RJ's excess falls in the spike months and 94 is on sellers in SP. Start with the SP → RJ route at peak times. BA's excess is mostly outside the spikes: a steady problem.

- RR (9 orders) and AP (8) are too small. By chance 1–2 of 27 states would sit outside the overall rate; 12 do, so states really differ.
- 951 labelled orders (6%) have no state, 741 of them because the customer row is missing.

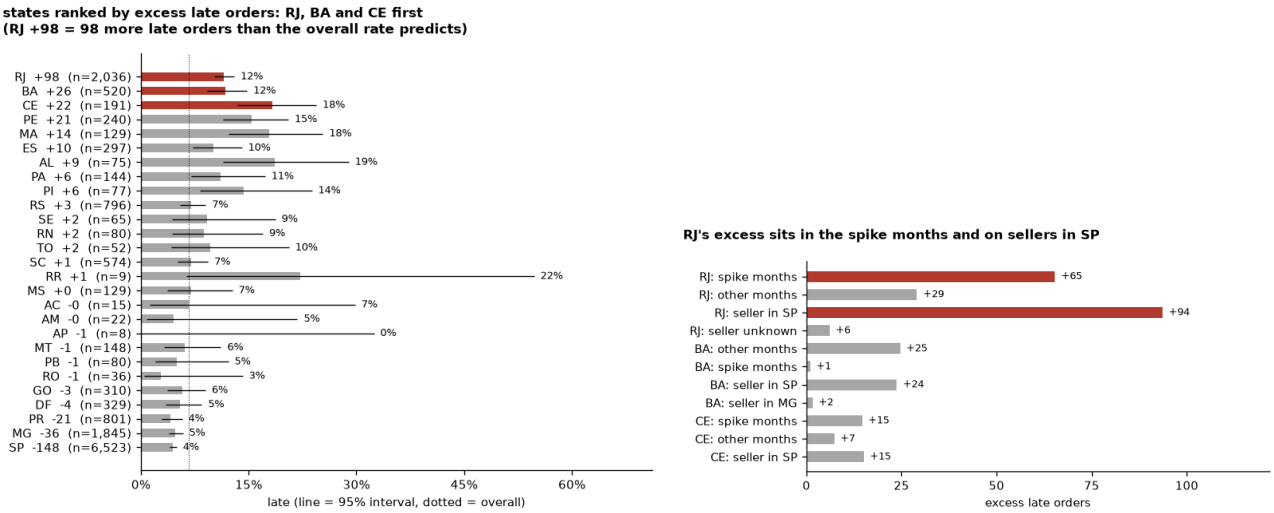


Figure 5: RJ adds the most late deliveries, mostly in the spike months and on sellers in SP.

3.4 Data quality, sensitivity and confidence

- Changing each cleaning choice keeps the late rate at 6.4–6.8%, and the top 3 states are always RJ, BA and CE.
- The real gap is coverage: 18% of orders have no label. The 944 cancelled orders could move the late rate from 6.4% (none late) to 11.8% (all late); 743 of them were already with the carrier, so 6.7% is best read as a lower bound.
- This is one modified extract (2016–2018): these are where to look first, not proven causes.

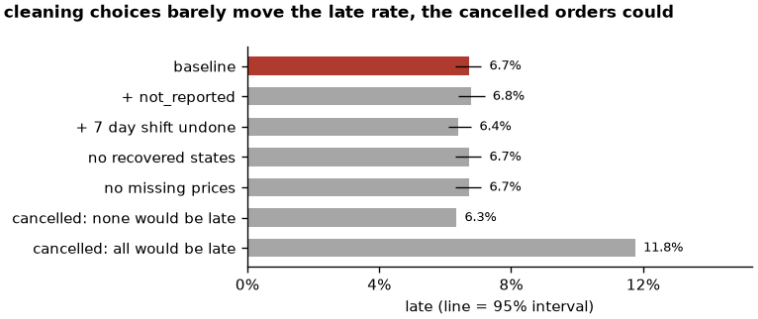


Figure 6: Cleaning choices barely move the late rate; the cancelled orders could move it far more.

4 Feature engineering

One row per labelled order (16,482). Columns not available at approval (payments, status, carrier and delivery dates, reviews, `shipping_limit_date`) are dropped *before* building, so using one by mistake fails. IDs and labels sit in `ids_labels.parquet`, same row order.

group	features	columns
timing	days approval to estimate, hours purchase to approval, purchase month / weekday / hour as sin + cos	8
basket	items, products, sellers, price total, freight total, freight share	6
product / seller	weight, volume, photos, seller states, share from the customer's state	5
missing flags	one per feature that can be missing	8
customer state	one-hot, 27 states + missing	28
category	most expensive item, top 15 + other + missing	17

- **Representation.** Sums per order, set to missing if a part is missing. Missing stays NaN with a flag; imputing here would use future test orders. Cyclic time as sin + cos, so December sits next to January; no year. Rare categories grouped as "other".
- **Kept.** Two copies dropped: `est_days_from_purchase` (= lead days + approval hours / 24) and `n_categories` (= not category missing). The highest remaining correlation is 0.78 (weight vs volume). Each feature's meaning and source columns: `feature_descriptions.csv`.
- **No leakage.** No source column is forbidden (checked in code). Each feature alone scores an AUC of 0.43–0.61, while the outcome `days_late` as a control gets 1.0.
- **Reliability.** A check model, run after step 4's resource numbers: a gradient-boosted tree [9] trained on all orders before a cutoff and tested on the months after, at three cutoffs (February, April, June 2018). All features score 0.58–0.62 every time, each 95% bootstrap interval above 0.5. Lead time alone scores 0.45–0.53, and the leak control 1.0. Modest but real signal, and no leak.

5 Reproducibility and resources

One command, strict checks. `run.py` (with uv, or pip and `requirements.txt`) stops unless:

1. Python is 3.12 and the supplied files match their SHA-256 hashes;
2. `data/` is rebuilt from nothing, so no earlier output survives;
3. every notebook assert passes, and every cell runs once, in order, with no error;
4. every output matches `expected.sha256` byte for byte.

A fresh clone and a clean Linux container without uv both pass. Every SQL query run is saved to `executed_sql.sql`. Environment: Python 3.12, pandas 3.0, pyarrow 26.

Estimated vs measured. Memory is peak RSS above the step's start, sampled every 10 ms; it moves between runs, so the table shows the range over 5 runs. Estimates from each step's in-memory data: memory 2–3×; zstd Parquet 0.2–0.35× of the tables written.

	data	memory MB		saved MB		
step	MB	estimate	measured	estimate	measured	seconds
0 setup db	18.3	33–52	205	18–27	24.4	< 5
1 load	12.9	26–39	69–73	2.6–4.5	3.3	0.2
2 clean	17.6	35–53	40–47	3.5–6.2	4.3	0.5–0.6
3 analysis	21.4	43–64	49–66	< 1	0.25	1.8–1.9
4 features	24.0	48–72	40–57	2.1–3.7	0.9	0.2–0.3

- Steps 2–4 land on or near their memory estimates (no run more than 20% outside). Step 1 needs about twice its estimate, likely Arrow buffers held before pandas wraps them (not measured separately).
- Saved storage lands inside the estimate for steps 0–2. Step 4 saved 0.9 MB against 2.1–3.7: one-hot zeros compress far better than the rule assumes. Temporary storage is 298 MB, only for the 10× engine comparison, then deleted.
- **Step 0 was the big miss** (205 vs 33–52 MB). The reviews contain emoji, and one character above U+FFFF makes Python store the whole 18-million-character text at 4 bytes per character

(73 MB) [8]. Measured: reading as bytes peaks at 33 MB, decoding at 123 MB, and reading as text (what `setup_db.py` does) at 196 MB. The loader is supplied, so it stays.

Limitations. Findings describe this modified extract; associations are not causes. 18% of orders are unlabelled, so the late rate is a lower bound.

Use of AI. I used Claude Code (Anthropic) as a coding assistant. It wrote and tested code with me, including the extra engine comparisons (DuckDB, Polars), helped write the LaTeX, checked for typos, and attacked the work against the rubric to find weak spots. I made the decisions, checked the results, and can explain each of them.

References

[1] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. https://doi.org/10.1080/01621459.1927.10502953

[2] L. D. Brown, T. T. Cai, A. DasGupta. Interval estimation for a binomial proportion. *Statistical Science*, 16(2):101–133, 2001. https://doi.org/10.1214/ss/1009213286

[3] SQLite. EXPLAIN QUERY PLAN. https://www.sqlite.org/eqp.html

[4] Apache Arrow. ADBC: Arrow Database Connectivity. https://arrow.apache.org/adbc/current/index.html

[5] M. Raasveldt, H. Mühleisen. DuckDB: an embeddable analytical database. *Proc. SIGMOD*, 1981–1984, 2019. https://doi.org/10.1145/3299869.3320212

[6] Polars user guide: streaming. https://docs.pola.rs/user-guide/concepts/streaming/

[7] Apache Parquet: compression. https://parquet.apache.org/docs/file-format/data-pages/compression/

[8] M. von Löwis. PEP 393: Flexible string representation, 2010. https://peps.python.org/pep-0393/

[9] F. Pedregosa et al. Scikit-learn: machine learning in Python. *JMLR*, 12:2825–2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html

[10] Olist. Brazilian E-Commerce Public Dataset (modified for this course), CC BY-NC-SA 4.0. https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce